

Lecture 8: Streaming and Sketching I

“In low-space, nobody can remember your stream.”

In previous chapters, we have seen that memory (*i.e.*, *space*) is a valuable resource, and can be a limiting factor when designing and running algorithms and data structures: and we have covered some concepts and general-purpose constructs, such as hash families, as well as data structures such as hash tables, Bloom filters, to reduce the amount of space used.

In the next two chapters, we will focus on an algorithmic paradigm where space is *the* core focus: *streaming algorithms*. In the streaming setting, the input itself is gigantic, too large to be stored in memory: and it is provided to the algorithm a little piece at a time, as a *stream* of updates. For instance, we could consider the MIN-CUT problem from a few chapters ago, except that now instead of being given the graph $G = (V, E)$ as input once and for all, the algorithms received E one edge at a time: every time step, a new edge $e \in E$ (in arbitrary order) is “revealed” to the algorithm, and at the end of the stream, when all is said and done, the algorithm must output a min-cut.

The good news is that in this streaming setting, the algorithm is *not* penalized for *time*: at each time step, it is allowed to run arbitrary computations.⁴⁵ The bad news is that the algorithm is penalized for *space*: it has very, very little memory at its disposal, typically much less than what would be needed to store the whole input itself. So *you’re allowed to run arbitrarily long computations, but you can’t afford to remember on what*.

⁴⁵ The algorithms we will see don’t do that, and are time-efficient. But technically, they *could*.

Streaming, at a glance. The goal is to solve a given computational task $\mathcal{T}$ on an (unknown) input x . This input x is provided to the algorithm as a *stream* σ of *updates*: that is, the t -th element of the stream σ is a *token* σ_t which provides a piece of the input. This stream comes in an arbitrary order (we do not assume the input is given “nicely” to the algorithm).

The algorithm has very small working memory, only s bits, and is allowed to make a number $p \geq 1$ of *passes* over this stream, typically $p = 1$ (in which case it only sees the stream once). After making these p passes over the input, the algorithm must output some solution to the task $\mathcal{T}$ on x , and be correct with high probability. Of course, this seems impossible: how to solve a problem

when you cannot even remember its whole input? And indeed, many things *are* provably impossible in the streaming setting. But quite surprisingly, many things *are* possible, and the techniques underlying them are incredibly powerful – and useful.

For additional content and details on streaming algorithms, including some of those seen in this chapter, the reader is referred to the (excellent) lecture notes by Amit Chakrabarti [AC], available at <https://www.cs.dartmouth.edu/~ac/Teach/data-streams-lecnotes.pdf>.

The Basic Setup

We will specifically focus on one-pass algorithms, unless specified otherwise. m denotes the length of the stream

$$\sigma = \langle a_1, \dots, a_m \rangle$$

where each a_i belongs to the universe $\mathcal{X}$ of size n .⁴⁶ We do not impose any bound on the time complexity of our algorithms, but we will enforce that they use very little memory (space), with space complexity denoted by s . We will aim for

$$s = o(\min(m, n))$$

and would love to use much less, ideally

$$s = O(\log m + \log n)$$

or, if not, $s = \text{poly}(\log m, \log n)$. To do so, we will allow for randomised algorithms *and* approximation algorithms, where the quality of the approximation will be controlled by a parameter $\epsilon > 0$, usually thought of as an (arbitrarily) small fixed constant.

The Majority Problem

To begin, consider the (seemingly) very simple question of deciding whether there is *one* element that appears at least half the time in the stream: that is, some $j \in [n]$ whose frequency f_j , defined as

$$f_j := \sum_{i=1}^m \mathbb{1}_{a_i=j}$$

satisfies $f_j \geq \frac{m}{2}$. This is the MAJORITY problem: at first glance, this seems very easy! Yet, spending some time thinking about it, you should convince yourself that it is surprisingly non-trivial to solve it using little memory.

The first algorithm we will see, due to Misra and Gries⁴⁷, is quite incredible in that regard: what it does is solving, *deterministically*, a related (and more general) version of this question, which

⁴⁶ Note that this notation is swapped with respect to the previous lectures, in order to be consistent with the streaming algorithms literature.

Of course, there are *at most* two such elements, and at most one if we define the question as “is there some j such that $f_j > m/2$?” The issue is that we do not know *a priori* which one(s) of the n elements could be the majority element(s).

⁴⁷ Jayadev Misra and David Gries. Finding repeated elements. *Sci. Comput. Program.*, 2(2):143–152, 1982

we will return into more detail in the next chapter: the question of *frequency estimation*, which asks to approximate the frequency f_j 's, not just decide which ones are at least $m/2$.

Theorem 40. *The MISRA-GRIES algorithm (Algorithm 15) is a deterministic one-pass algorithm which, for any given parameter $\varepsilon \in (0, 1]$, provides $\hat{f}_1, \dots, \hat{f}_n$ of all element frequencies such that*

$$f_j - \varepsilon m \leq \hat{f}_j \leq f_j, \quad j \in [n]$$

with space complexity $s = O(\log(mn)/\varepsilon)$. (In particular, it can be used to solve the MAJORITY problem in two passes.)

An interesting observation here is that of course the algorithm cannot compute explicitly n values $\hat{f}_1, \dots, \hat{f}_n$: this by itself would take $\Omega(n)$ space. What it does is *implicitly* do so, by only storing the values $\hat{f}_j$'s that are *non-zero* (and making sure there are very few of them, only $O(1/\varepsilon)$). Which makes sense, since we should not have many more non-zero estimates than this: after all, there can only be at most $1/\varepsilon$ element $j \in [n]$ such that $f_j \geq \varepsilon m$ (i.e., which appear at least an ε fraction of the time in the stream)!

Algorithm 15: The MISRA-GRIES algorithm. Only store in A : if $A[j]$ does not exist, it is 0. Instead of a BST, one could use a linked list, for instance: this would have the same space complexity, but a larger update time at each step.

Input: Parameter $\varepsilon \in (0, 1]$

```

1:  $A \leftarrow \emptyset$   $\triangleright$  Use, e.g., a self-balancing binary search tree (BST)
2: Set  $k \leftarrow \lceil 1/\varepsilon \rceil$ 
3: for all  $1 \leq i \leq m$  do
4:   Get item  $a_i = j \in [n]$ 
5:   if  $A[j] > 0$  then  $\triangleright j$  is in the BST
6:      $A[j] \leftarrow A[j] + 1$ 
7:   else if  $A[j] = 0$  and  $|A| < k - 1$  then
8:      $A[j] \leftarrow 1$ 
9:   else if  $A[j] = 0$  and  $|A| = k - 1$  then
10:    for all  $j' \in A$  do  $\triangleright$  Loop over all  $j'$  such that  $A[j'] > 0$ 
11:       $A[j'] \leftarrow A[j'] - 1$   $\triangleright$  If  $A[j']$  reaches 0, remove it from  $A$ 
```

Output: On query $j \in [n]$, **return** $A[j]$

Proof. First, note that since A never stores more $k = O(1/\varepsilon)$ elements, each of them taking $O(\log n + \log m)$ bits (for the index of the element, and its current count), the space s is bounded as

$$s = O\left(\frac{\log(mn)}{\varepsilon}\right)$$

as claimed.

To prove correctness, fix any $j \in [n]$. Since $A[j]$ can only be incremented (on Line 8 or 6) when element j appears in the stream, we have, at the end of the stream,

$$\hat{f}_j = A[j] \leq f_j.$$

For the other inequality (the lower bound), we need to get a grasp on the number of times $A[j]$ is decremented, which can only happen in Line 11. Every time this line is reached, this means that

(1) nothing else happens in this step (no increment to $A[j]$), and
 (2) exactly $k - 1$ other counters are decremented (in the **for all** loop).

We can see (1), conceptually, as one increment to $A[j]$ immediately followed by a decrement to $A[j]$: thinking of it this way allows us to say that $A[j]$ is incremented every time j appears in the stream – but sometimes, it is decremented immediately after as well, and lets us combine (1) and (2) to say that every time Line 11 is reached, *exactly* k decrements are performed in A . Given that every increment uniquely corresponds to one of the m steps, this means that each execution of Line 11 corresponds to a disjoint chunk of k steps: when the increments to the $A[j']$'s had happened. But there are only m steps in total, so if each decrement “burns” k of them, there can be at most $\frac{m}{k}$ decrements steps! This shows that

$$\hat{f}_j = A[j] \geq f_j - \frac{m}{k},$$

which, given the value of k , implies $\hat{f}_j \geq f_j - \varepsilon m$.

To see how the “In particular” statement follows, consider applying the MISRA-GRIES algorithm with $\varepsilon = 1/4$, and at the end of the first pass considering the set $S \subseteq [n]$ of elements for which $\hat{f}_j \geq \frac{1}{4}m$. If j^* is a majority element, then

$$\hat{f}_j \geq f_j - \varepsilon m \geq \frac{1}{2}m - \frac{1}{4}m = \frac{1}{4}m,$$

so $j^* \in S$. Conversely, if $j \in S$, then

$$f_j \geq \hat{f}_j \geq \frac{1}{4}m,$$

and so there can be at most $\frac{m}{(1/4)m} = 4$ elements in S . So keeping S in memory only takes $4 \cdot O(\log n) = O(\log n)$ bits. Then, all that remains to do is, in the *second* pass, count *exactly* the number of times each elements $j \in S$ appears, and check if that's at least $m/2$. Each such counter takes $O(\log m)$ bits, and there are only (at most) 4 counters to maintain now. $\square$

A short (yet important) detour: frequency estimation

We have defined above the *frequency* f_j of an element $1 \leq j \leq n$ of the universe (for a given stream σ). If we look at this as a *frequency vector*

$$\mathbf{f} = (f_1, \dots, f_n),$$

frequency vector

we can define, for each $p \in (0, \infty)$, its p -moment as

$$F_p(\sigma) = \|\mathbf{f}\|_p = \left(\sum_{j=1}^n f_j^p \right)^{1/p}$$

and extend it to $p \in \{0, \infty\}$ by taking limits:

$$F_0(\sigma) = \sum_{j=1}^n \mathbb{1}_{\{f_j > 0\}}$$

(the number of non-zero frequencies, *i.e.*, number of distinct elements in the stream) and

$$F_{\infty}(\sigma) = \max_{1 \leq j \leq n} f_j$$

(the maximum frequency). For simplicity, the explicit dependence on the stream σ is typically omitted, so we just write F_p for the p -frequency moment, for $p \in [0, \infty]$.

frequency moments

Why do we care about these frequency moments? It turns out they encode *a lot* of useful information about the stream: as said above, F_0 is the *number of distinct elements*:

$$1 \leq F_0 \leq n$$

which measures the “coverage” of the stream; while

$$F_1 = \sum_{j=1}^n f_j$$

is just the number of elements (in particular, in our “basic setting” where each stream item a_i corresponds to one element, $F_1 = m$), and F_{∞} tells us the frequency of the most frequent element (in this basic setting, again, $1 \leq F_{\infty} \leq m$). The second moment, F_2 , is also particularly important, as it corresponds to how “uniformly distributed” the stream elements are: think of it as some sort of empirical variance.

It should not be a surprise, then, that a lot of time and effort has been dedicated to designing good streaming algorithms to get *good (multiplicative) estimates of the moments of the stream*, especially F_0 and F_2 . We will see a few!

F_p estimation

Cash register and turnstile models. Before doing so, one last subtlety: in the “basic setting” mentioned above, each stream token a_i is simply an element $j_i \in [n]$ of the universe, “revealed” to the algorithm. This can be generalized to allow for *multiplicities*: the i -th token is a pair $a_i = (j_i, c_i)$, which tells the algorithm to include an additional $c_i \geq 0$ copies of element $j_i \in [n]$. This is called the *cash register model*, where the stream is a sequence of positive updates to element counts, and, at the end of the stream,

$$f_j = \sum_{\substack{i \in [m] \\ j_i = j}} c_i, \quad j \in [n]$$

With this notation, the “basic setting” is just the cash register model when all c_i ’s are equal to 1.

But we don’t have to stop here! In the most general setting, the *turnstile model*,⁴⁸ each stream token is an update which can be either positive or negative: that is, $a_i = (j_i, c_i)$, where $c_i \in \mathbb{Z}$ (during the i -th update, we can add or remove copies of j_i). This turnstile model is the most challenging, algorithmically, as “numbers can go up or down” and (intuitively) it makes it more challenging to keep track of their current count

⁴⁸ There is a third model, the *strict turnstile model*, which is the same as the turnstile one, with the added restriction that the current running count of any element $j \in [n]$ cannot be negative. So if I currently have a total count of 10 for element $j = 23$ at this point of the stream, the next token can remove up to 10 copies of element 23; but not more.

The Approximate Counting Problem

Recall that at the end of the proof of Theorem 40, we mentioned in passing we could use counters, an obvious building block barely worth mentioning and thinking about: a counter is just a number, which we increment. In this section, we will describe and analyse the streaming version of this primitive, which turns out to be surprisingly interesting and non-trivial. This is the Morris Counter algorithm, due to, well, Morris⁴⁹, which provides a constant-factor estimate of the number of elements of the stream: that is, an F_1 estimator.⁵⁰ Another way to think of it, which is the viewpoint we will be taking here, is to consider the task of counting “important” events: at each time step $1 \leq t \leq m$, we are told if some event happened ($a_i = 1$) or not ($a_i = 0$): the goal is to estimate how many events happened in total, *i.e.*, the number $d = \sum_{i=1}^m a_i$.

The algorithm is described below: at a high-level, what it does is to keep track of the *exponent* of the number of things to count, instead of the count itself. Which makes sense: if we succeed in doing so, this is a 2-approximation of the actual count. But how to keep track of its exponent without keeping track of the number itself? When do we know when to increment that exponent? The key here is to do it probabilistically: if the current estimate of the count is 2^x , then upon seeing a new “important event” we want to increment x with *some* probability. Since incrementing x means multiplying 2^x by 2, this probability should become smaller the bigger x gets since going from $x = 0$ to $x = 1$ is increasing our estimate of the count from $2^0 = 1$ to $2^1 = 2$, but going from $x = 10$ to $x = 11$ is increasing our estimate from 1024 to 2048: a much larger jump). So the larger x is, the less likely it should be to increment it when seeing a new event. How much less likely? It turns out that the “right” scaling to keep a correct count *in expectation* is, upon seeing a new “important event”, to increment x with probability $1/2^x$.

```

1:  $x \leftarrow 0$ 
2: for all  $1 \leq i \leq m$  do
3:   Get item  $a_i \in \{0, 1\}$ 
4:   if  $a_i = 1$  then
5:      $r_i \leftarrow \text{Bern}(1/2^x)$        $\triangleright$  Independent of previous choices.
6:      $x \leftarrow x + r_i$ 
7: return  $\hat{d} \leftarrow 2^x - 1$ 

```

The space complexity is a little annoying to bound: we *expect* x to never exceed $\log_2 m$, since d is at most m by definition and we should have $2^x \approx d$. But there is a very, very small chance that all Bernoullis turn out to be 1, in which case x could become as big as m ! This would make no sense, and also mean we would need $O(\log m)$ bits to store x , exactly what we do not want to pay. *However*, one can show that with overwhelming probability x remains at most $O(\log m)$, and so the space complexity required is only

Chapter 4 of [AC]

⁴⁹ Robert H. Morris Sr. Counting large numbers of events in small registers. *Commun. ACM*, 21(10):840–842, 1978

⁵⁰ In our “basic setting” where $F_1 = m$, an F_1 estimator only makes sense if we don’t know the length m of the stream in advance: which is why we will instead be looking at Morris counters for the related, but different task described below.

Algorithm 16: The MORRIS COUNTER algorithm.

If x ever exceeds this value, the algorithm can just abort: this only adds a vanishing small amount to the probability of error.

$s = O(\log \log m)$.

The proof of correctness of Algorithm 16 relies on the key lemma below, analysing the expectation and variance of $\hat{d}$:

Lemma 40.1. *The random variable $\hat{d}$ defined in Algorithm 16 satisfies*

$$\mathbb{E}[\hat{d}] = d$$

and

$$\text{Var}[\hat{d}] = \frac{d(d-1)}{2}$$

Proof. Define C_i , for $1 \leq i \leq m$, as the value of 2^x in Algorithm 16 at the end of step i ; so that $C_0 = 2^0 = 1$ and $\hat{d} = C_m - 1$.

For any $1 \leq i < m$, we then have

$$C_{i+1} = \begin{cases} 2 \cdot C_i & \text{if } a_{i+1} = 1 \text{ and } r_{i+1} = 1 \\ C_i & \text{otherwise} \end{cases}$$

which we can rewrite as $C_{i+1} = (1 + a_{i+1}r_{i+1})C_i$. Recalling that $r_{i+1} \sim \text{Bern}(1/C_i)$ gives us

$$\mathbb{E}[C_{i+1} \mid C_i] = (1 + a_{i+1}\mathbb{E}[r_{i+1} \mid C_i]) \cdot C_i = \left(1 + \frac{a_{i+1}}{C_i}\right) \cdot C_i = C_i + a_{i+1}$$

and, by the Law of Total Expectation,

$$\mathbb{E}[C_{i+1}] = \mathbb{E}[\mathbb{E}[C_{i+1} \mid C_i]] = \mathbb{E}[C_i] + a_{i+1}.$$

This gives us

$$\mathbb{E}[C_m] = \mathbb{E}[C_0] + \sum_{i=0}^{m-1} (\mathbb{E}[C_{i+1}] - \mathbb{E}[C_i]) = 1 + \sum_{i=0}^{m-1} a_{i+1} = 1 + d$$

showing that $\mathbb{E}[\hat{d}] = d$. The above actually showed the more general statement that

$$\mathbb{E}[C_i] = 1 + \sum_{j=1}^i a_j, \quad 1 \leq i \leq m, \quad (58)$$

which we will use very soon.

To compute the variance, we similarly analyse $\mathbb{E}[C_m^2]$: For any $1 \leq i < m$,

$$\begin{aligned} \mathbb{E}[C_{i+1}^2 \mid C_i] &= \mathbb{E}[(1 + a_{i+1}r_{i+1})^2 \mid C_i] \cdot C_i^2 \\ &= \left(1 + \frac{a_{i+1}(2 + a_{i+1})}{C_i}\right) \cdot C_i^2 \\ &= C_i^2 + a_{i+1}(2 + a_{i+1})C_i \end{aligned}$$

where the second equality follows from expanding the square and computing the expectation. Again, by the Law of Total Expectation,

$$\begin{aligned} \mathbb{E}[C_{i+1}^2] &= \mathbb{E}[\mathbb{E}[C_{i+1}^2 \mid C_i]] = \mathbb{E}[C_i^2] + a_{i+1}(2 + a_{i+1})\mathbb{E}[C_i] \\ &= \mathbb{E}[C_i^2] + a_{i+1}(2 + a_{i+1})\left(1 + \sum_{j=1}^i a_j\right) \quad (\text{By Eq. (58)}) \\ &= \mathbb{E}[C_i^2] + 3a_{i+1} \sum_{j=1}^{i+1} a_j \end{aligned}$$

where that last step is completely magical, but “immediate in hindsight” by checking the two possible cases: $a_{i+1}(2 + a_{i+1})\left(1 + \sum_{j=1}^i a_j\right) = 3a_{i+1}\left(a_{i+1} + \sum_{j=1}^i a_j\right)$ for both $a_{i+1} = 0$ and $a_{i+1} = 1$. This gives us

$$\begin{aligned}\mathbb{E}\left[C_m^2\right] &= \mathbb{E}\left[C_0^2\right] + 3 \sum_{i=0}^{m-1} a_{i+1} \sum_{j=1}^{i+1} a_j = 1 + 3 \sum_{i=1}^m \sum_{j=1}^i a_i a_j \\ &= 1 + 3 \cdot \frac{1}{2} \left(\left(\sum_{i=1}^m a_i \right)^2 + \sum_{i=1}^m a_i^2 \right) \\ &= 1 + 3 \cdot \frac{1}{2} (d^2 + d)\end{aligned}$$

(recalling, for the last step, that $a_i^2 = a_i$ for all i , since $a_i \in \{0, 1\}$). Since $\mathbb{E}[C_m]^2 = d + 1$, we finally get

$$\text{Var}[C_m] = 1 + 3 \cdot \frac{1}{2} (d^2 + d) - (d + 1)^2 = \frac{d^2 - d}{2},$$

as claimed. $\square$

Is this good? Unfortunately, the above is not particularly useful, as is: our bound on the variance is $O(d^2)$, the expectation is $\Theta(d^2)$, so applying Chebyshev’s inequality would only give us a constant-factor estimate (e.g., up to a factor 2, or 10), with constant probability. We *could* then improve the probability of success via the median trick, but that would not improve the quality of the estimate: we would still only get a quite bad constant-factor approximation, but now with high probability.

To get an $(1 + \varepsilon)$ -approximation with high probability, we need to obtain a more accurate estimate *before* applying the median trick. To do so, if we want to use Chebyshev (and we do!), we need to *reduce the variance*. But the computation above is quite tight, we cannot hope to significantly improve our bound on the variance! So *how do we do this?*

By averaging! Specifically, if we have k independent (or even pairwise independent) copies $X_1, \dots, X_k$ of some random variable with finite mean μ and variance σ^2 , then their average

$$\bar{X} := \frac{1}{k} \sum_{\ell=1}^k X_\ell$$

has the same mean $\mathbb{E}[\bar{X}] = \mu$, but *much lower* variance:

$$\begin{aligned}\text{Var}[\bar{X}] &= \text{Var}\left[\frac{1}{k} \sum_{\ell=1}^k X_\ell\right] = \frac{1}{k^2} \text{Var}\left[\sum_{\ell=1}^k X_\ell\right] = \frac{1}{k^2} \sum_{\ell=1}^k \text{Var}[X_\ell] \\ &= \frac{1}{k^2} \cdot k\sigma^2 = \frac{\sigma^2}{k}\end{aligned}$$

This gives a general recipe: (1) start an estimator with the right expectation and a not-so-great variance, (2) build an estimator with better variance by averaging k independent copies (now you can

median-of-means

use Chebyshev!), getting better accuracy; and (3) take the median of T independent copies of that new *averaged*, higher-accuracy estimator to amplify its success probability (median trick). This is the *median-of-means* technique, and picking k and T suitably leads to a high-accuracy, high-probability estimate – at the cost of a factor kT in the amount of resources used by the original estimator of (1).

In our case, averaging $k = O(1/\epsilon^2)$ independent copies of the Morris Counter enables us to bring down the variance by this factor, leading to a $(1 + \epsilon)$ -estimate with high (constant) probability. Using the median trick afterwards (running $T = O(\log(1/\delta))$ copies of this improved-variance algorithm, and taking the median result) gives a high-probability result, leading to the following:

Theorem 41. *The medians-of-means version of the MORRIS COUNTER is a randomised one-pass algorithm which, for any given parameters $\epsilon, \delta \in (0, 1]$, provides an estimate $\hat{d}$ of the number d of non-zero elements of the stream such that*

$$\Pr \left[(1 - \epsilon)d \leq \hat{d} \leq (1 + \epsilon)d \right] \geq 1 - \delta$$

with space complexity

$$s = O \left(\frac{\log \log m}{\epsilon^2} \cdot \log \frac{1}{\delta} \right)$$

that is, doubly logarithmic in m .

Again, maintaining an exact counter would take $O(\log m)$ bits: this is an exponential improvement! And yet, *we can do better*. Instead of using the median-of-means technique, we can be more careful about the algorithm itself: where we incremented x with probability $1/2^x$ and returned $2^x - 1$, we will, for a suitable choice of $\alpha = \alpha(\epsilon, \delta) > 0$, increment it with probability $1/(\alpha(1 + \alpha)^x)$ and return $(1 + \alpha)^x - 1$. We will see the details in the tutorial, leading to this (much) improved bound:

Theorem 42. *The “careful” version of MORRIS COUNTER is a randomised one-pass algorithm which, for any given parameters $\epsilon, \delta \in (0, 1]$, provides an estimate $\hat{d}$ of the number d of non-zero elements of the stream such that*

$$\Pr \left[(1 - \epsilon)d \leq \hat{d} \leq (1 + \epsilon)d \right] \geq 1 - \delta$$

with space complexity

$$s = \log \log m + O \left(\log \frac{1}{\epsilon} + \log \frac{1}{\delta} \right)$$

that is, doubly logarithmic in m and logarithmic in $1/\epsilon$.

The Distinct Elements Problem

We start this section with the TIDEMARK algorithm, due to Alon, Matias and Szegedy (AMS), which provides a constant-factor estimate of the number of distinct elements of the stream: that is, an

F_0 estimator. In this section, we define d as this number of distinct elements, *i.e.*,

$$d = \sum_{j \in [n]} \mathbb{1}_{f_j > 0}$$

In what follows, for a given positive integer k , $\text{zeros}(k)$ denotes the largest power of 2 which divides k , or, equivalently, the number of trailing zeroes in the binary representation of k . The space

```

1: Pick  $h: [n] \rightarrow [n]$  from a strongly universal hashing family
2:  $z \leftarrow 0$ 
3: for all  $1 \leq i \leq m$  do
4:   Get item  $a_i \in [n]$ 
5:   if  $\text{zeros}(h(a_i)) \geq z$  then
6:      $z \leftarrow \text{zeros}(h(a_i))$ 
7: return  $\hat{d} \leftarrow \sqrt{2} \cdot 2^z$ 

```

Algorithm 17: The TIDEMARK algorithm

complexity of Algorithm 17 is

$$s = O(\log n)$$

as all that is needed is storing the hash function ($O(\log n)$ bits, for a suitable strongly universal hash family) and $1 \leq z \leq \log_2 n$ (which takes $O(\log \log n)$ bits). To analyse the correctness of the algorithm (and the quality of the estimate it outputs), define the random variables

$$Y_r := \sum_{\substack{j \in [n] \\ f_j > 0}} \mathbb{1}_{\text{zeros}(h(j)) \geq r}, \quad r \geq 0$$

(where the randomness is over the choice of the hash function h). One can check that, by definition,

$$Y_r \geq 1 \Leftrightarrow z \geq r$$

for every integer $r \geq 0$. Moreover,

$$\mathbb{E}[Y_r] = \sum_{\substack{j \in [n] \\ f_j > 0}} \Pr[\text{zeros}(h(j)) \geq r] = \sum_{\substack{j \in [n] \\ f_j > 0}} \frac{1}{2^r} = \frac{d}{2^r}$$

where we used the fact that each $h(j)$ is uniformly distributed to write that $\Pr[\text{zeros}(h(j)) \geq r] = \Pr[2^r \text{ divides } h(j)] = \frac{1}{2^r}$. Similarly, using pairwise independence,

$$\text{Var}[Y_r] = \sum_{\substack{j \in [n] \\ f_j > 0}} \text{Var}[\mathbb{1}_{\text{zeros}(h(j)) \geq r}] \leq \sum_{\substack{j \in [n] \\ f_j > 0}} \frac{1}{2^r} = \frac{d}{2^r}$$

the inequality using $\text{Var}[X] \leq \mathbb{E}[X^2]$ and the fact that $X^2 = X$ when X is an indicator random variable. Using these two facts, for every $r \geq 0$,

- $\Pr[z \geq r] = \Pr[Y_r \geq 1] \leq \mathbb{E}[Y_r] = \frac{d}{2^r}$ by Markov;

- $\Pr[z \leq r] = \Pr[Y_{r+1} = 0] \leq \frac{\text{Var}[Y_{r+1}]}{\mathbb{E}[Y_{r+1}]^2} \leq \frac{2^{r+1}}{d}$ by Chebyshev,

using that $\Pr[Y_{r+1} = 0] \leq \Pr[|Y_{r+1} - \mathbb{E}[Y_{r+1}]| \geq \mathbb{E}[Y_{r+1}]]$. This is all we need! Setting $C := 3\sqrt{2}$,

$$\Pr[\hat{d} \geq C \cdot d] \leq \Pr\left[z \geq \left\lceil \log_2(C \cdot d / \sqrt{2}) \right\rceil\right] \leq \frac{\sqrt{2}d}{Cd} = \frac{1}{3}$$

while

$$\Pr[\hat{d} \leq d/C] \leq \Pr\left[z \leq \left\lfloor \log_2(d / (\sqrt{2}C)) \right\rfloor\right] \leq \frac{2d}{\sqrt{2}Cd} = \frac{1}{3}$$

Combining the above with the median trick, we readily get:

Theorem 43. *The (median trick version of the) TIDEMARK (AMS) algorithm is a randomised one-pass algorithm which, for any given parameter $\delta \in (0, 1]$, provides an estimate $\hat{d}$ of the number d of distinct elements of the stream such that, for some absolute constant $C > 0$,*

$$\Pr\left[\frac{1}{C} \cdot d \leq \hat{d} \leq C \cdot d\right] \geq 1 - \delta$$

with space complexity

$$s = O\left(\log n \cdot \log \frac{1}{\delta}\right).$$

This is not bad, but can we achieve estimation factor arbitrarily close to one, say, $1 + \varepsilon$? The answer is yes: the following algorithm, due to Bar-Yossef, Jayram, Kumar, Sivakumar and Trevisan (BJKST), does exactly that.

TODO See lecture for the analysis.

Algorithm 18: The BJKST algorithm

Input: Parameter $\varepsilon \in (0, 1]$

- 1: Set $k \leftarrow O(\log^2 n / \varepsilon^4)$, $T \leftarrow \Theta(1/\varepsilon^2)$
 - 2: Pick $h: [n] \rightarrow [n]$ from a strongly universal hashing family
 - 3: Pick $g: [n] \rightarrow [k]$ from a strongly universal hashing family
 - 4: $z \leftarrow 0$, $B \leftarrow \emptyset$
 - 5: **for all** $1 \leq i \leq m$ **do**
 - 6: Get item $a_i \in [n]$
 - 7: **if** $\text{zeros}(h(a_i)) \geq z$ **then**
 - 8: $B \leftarrow B \cup \{(g(a_i), \text{zeros}(h(a_i)))\}$
 - 9: **while** $|B| \geq T$ **do**
 - 10: $z \leftarrow z + 1$
 - 11: Remove every (a, b) with $b < z$ from B
 - 12: **return** $|B| \cdot 2^z$
-

Theorem 44. *The (median trick version of the) BJKST algorithm is a randomised one-pass algorithm which, for any given parameters $\varepsilon, \delta \in (0, 1]$, provides an estimate $\hat{d}$ of the number d of distinct elements of the stream such that, for some absolute constant $C > 0$,*

$$\Pr\left[(1 - \varepsilon) \cdot d \leq \hat{d} \leq (1 + \varepsilon)d\right] \geq 1 - \delta$$

with space complexity

$$s = O\left(\left(\log n + \frac{\log(1/\varepsilon) + \log \log n}{\varepsilon^2}\right) \cdot \log \frac{1}{\delta}\right).$$

This is pretty good, but... *Is it optimal?*

As it turns out, *no*. One can do slightly better, using a more involved algorithm and analysis, to obtain the optimal

$$s = O\left(\left(\log n + \frac{1}{\varepsilon^2}\right) \cdot \log \frac{1}{\delta}\right)$$

space complexity for F_0 estimation.⁵¹ We will not see it here, as this is beyond the scope of this course.

⁵¹ Daniel M. Kane, Jelani Nelson, and David P. Woodruff. An optimal algorithm for the distinct elements problem. In *PODS*, pages 41–52. ACM, 2010.